Мова: English Italiano **Українська**

Посібник h4nb

Пряме N-body моделювання на GPU NVIDIA: Hermite 4-го порядку, блокові часові кроки, FP64

> ✅ = уже працює в `main`. 🚧 = погоджений дизайн, ще не реалізовано (номер задачі в дужках).

Зміст

1. Проблема
2. Підходи
3. Архітектура
4. Швидкий старт
5. Конфігурація
6. Шпаргалка
7. Типові проблеми
8. Обладнання
9. Документи

1. Проблема

Дослідник вивчає диск або скупчення навколо одного масивного тіла: хмари газу навколо активного ядра галактики (АЯГ) або зорі навколо центральної маси. Сумарна маса частинок може становити від 10 % до 100 % центральної маси, тому центральне тіло рухається.

- **Точні сили:** кожна частинка притягує всі інші. Один повний розрахунок сил для N = 1 024 000 — це приблизно 10^{12} пар.
- **Контрольована точність:** максимальна відносна похибка енергії за один середній орбітальний період не перевищує 1e-8. Це має показати виміряний запуск.
- **Швидкість:** великі запуски тривають години на серверних GPU.
- **Відтворюваність:** ті самі вхідні дані, конфігурація, пристрої та збірка дають побітово той самий результат.

2. Підходи

Тема	Рішення	Чому
Інтегратор	Hermite 4-го порядку: прогноз, обчислення a та jerk, корекція	4-й порядок з одним розрахунком сил на крок
Часові кроки	Критерій Aarseth (η), блокові кроки степенями двійки, цілі ticks	Нові сили отримують лише активні частинки; точні часи блоків
Центральне тіло	Частинка 0, повноправний учасник	Зберігає імпульс
Точність	Лише FP64, без fast-math, без float-атоміків, фіксований порядок сум	Точність і побітово однакові запуски

Тема	Рішення	Чому
Softening	Необов'язковий Plummer ε; пресети АЯГ 0.01, зоряні 0.0001	Змінює модель; не приховує похибки
Кілька GPU	Розподілені джерела (j-parallel); власник додає часткові суми в порядку	Мало активних частинок на крок; однакова робота на GPU
Кілька вузлів 🚧	Один MPI rank на GPU, CUDA-aware MPI (T029–T036)	Ціль: 4–8 GH200
η	Вимірювати для кожної системи; ≈5e-4 для 1e-8	Діапазон T3 0.001–0.02 не досягає 1e-8
Оптимізація	Кожна оптимізація потребує виміру, який показує потребу	Припущення про швидкість GPU часто хибні

3. Архітектура

Один публічний заголовок (`include/h4nb/h4nb.hpp`) і один клас, `Simulation` . В API немає CUDA-типів. GPU-ядра та CPU-оракул тестів використовують один математичний заголовок (`src/hermite.cuh`).

Один блоковий крок:

1. Знайти наступний час блоку Tn = min(next).
2. Спрогнозувати всі частинки до Tn.
3. Вибрати активні частинки (next == Tn).
4. Надіслати активні цілі на кожну GPU.
5. Кожна GPU обчислює часткові a та jerk своїх джерел для всіх цілей (тайли в shared memory, один потік на ціль).
6. Надіслати кожну частину GPU-власнику.
7. Власник додає часткові суми в порядку пристроїв, коригує і вибирає новий крок.

Пам'ять: приблизно 0,4 ГБ на GPU для 1M частинок. Обмеження — обчислення.

4. Швидкий старт

Збірка (Linux x86_64 або aarch64, CUDA ≥ 12.0, CMake ≥ 3.24, GPU з compute capability ≥ 7.0):

```
make build                                    # Release, CMAKE_CUDA_ARCHITECTURES=native
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=89      # RTX 40
70 Ti
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES=90      # GH200
/ H100
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DCMAKE_CUDA_ARCHITECTURES="89;90" # both
cmake --build build -j
```

Запуск:

```
build/h4nb_mkdisk n=127999 frac=0.1 out=disk128k.snap     # prints mean / median / mass-we
ighted period
build/h4nb_run ic=disk128k.snap out=run128k t_end=6.3125 diag=0.0625 eta=0.0005 devices=0,
1
build/h4nb_order e=0.5                                     # Kepler orbit + 4th-order chec
k
build/h4nb_bench n=16384,131072,1024000 devices=0,1        # speed table
build/h4nb_study ic=disk128k.snap out=study diag=0.0625 etas=0.001,0.0005,0.00025 devices=
0,1   # eta study
```

`t_end`, `diag` і `snap` мають бути кратними `dt_max` (за замовчуванням 1/64), а `t_end` має ділитися на `diag`.

5. Конфігурація

✅ Ключі `h4nb_run` (файл, середовище або командний рядок, `key=value`):

Ключ	За замовчуванням	Значення
`ic`	—	початкові умови (текст або .snap)
`out`	—	папка результатів
`t_end`	—	кінцевий час
`eta`	`0.01`	точність Aarseth (≈0.0005 для 1e-8)
`eta_init`	`0.01`	множник першого кроку
`dt_max`	`1/64`	найбільший крок (рівень 0)
`max_level`	`40`	dt_min = dt_max / 2^max_level
`eps`	`0`	довжина softening ε
`mode`	`block`	block або fixed
`fixed_level`	`0`	рівень у режимі fixed
`fixed_center`	`0`	1: центр нерухомий (тести)
`devices`	`0`	список GPU, напр. 0,1
`diag`	`dt_max`	інтервал записів енергії
`snap`	`0`	інтервал знімків (кратний diag)
`r_close`	`-1`	радіус близьких пар; 0 = вимкнено
`n`	`0`	очікувана кількість частинок (0 = без перевірки)
`central_id`	`0`	ID запису 0 (центральне тіло)
`period`	`0`	опорний період (0 = оцінка, потрібно G = 1)
`period_kind`	`mean`	mean, median або mass_weighted

Ключ	За замовчуванням	Значення
`preset`	`none`	none, agn (ε 0.01), stellar (ε 0.0001)
`G`	`1`	гравітаційна стала
`dissipation`	`off`	поки лише off
`tile`	`128`	цілей на блок (поки лише 128)
`blocks_per_sm`	`4`	від 1 до 16
`snap_times`	—	явні часи знімків
`energy_tol`	`1e-8`	поріг вердикту енергії
`perf`	`1`	0: без performance.dat

✅ П'ять джерел. Пізніше джерело перемагає:

1. значення за замовчуванням
2. `preset=none|agn|stellar`
3. `config=<file>` (рядки key=value)
4. змінні середовища `H4NB_<KEY>`
5. командний рядок

Невідомий ключ або хибне значення — це помилка (код виходу 2). Кожен запуск записує `config.used`. `help=1` друкує всі ключі.

🚧 Ще заплановано	Задача
`dissipation=linear_drag, drag_tau, drag_vbg`	T041–T043
`tile=64, tile=256`	T017, T018
`device_offset, mpi_host_staging`	T030, T032
`sizes, tiles, bpsm, fp64_peak`	T019

Як вибрати основні значення

`eta`	Виміряйте через дослідження η. Почніть з ≈0.0005. Вартість росте як 1/√η.
`dt_max`	Від 1/64 до 1/16 найкоротшого періоду, що вас цікавить.
`max_level`	Збільште, якщо бачите попередження dt < dt_min.
`eps`	0 для точкових мас; preset=agn або preset=stellar для двох типів моделей.
`diag`	Одна перевірка енергії коштує $N^2/2$ пар. Для 1M беріть T/16 або більше.
`tile, blocks_per_sm` 🚧	Виміряйте через h4nb_bench; збережіть найкращі значення в конфігу машини.
`GPU / вузли`	Задає лаунчер (devices=, mpirun, srun). Розмір кластера в коді не зафіксовано.

6. Шпаргалка

Файл конфігурації ✅

```
# configs/agn.cfg
preset=agn                    # eps = 0.01
eta=0.0005                    # measured value for 1e-8
dt_max=0.015625
diag=0.0625
period=6.283185307179586      # 2*pi for G = 1, M = 1, a = 1
```

Запуск з файлом конфігурації ✅

```
build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run1 t_end=6.3125 devices=0,1
H4NB_ETA=0.00025 build/h4nb_run config=configs/agn.cfg ic=disk128k.snap out=run2 t_end=6.3
125
build/h4nb_run config=run1/config.used out=run1_repeat     # repeat a run exactly
```

Кілька вузлів 🚧

```
srun --ntasks-per-node=4 --gpus-per-task=1 --gpu-bind=single:1 \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
mpirun -np 8 --map-by ppr:4:node --bind-to core \
  build/h4nb_run config=configs/gh200.cfg ic=disk1m.snap out=run1m t_end=6.3125
```

C++ ✅

```
#include <cmath>
#include <h4nb/h4nb.hpp>

h4nb::Particles p = h4nb::read_snapshot("disk128k.snap");   // or h4nb::load_text("ic.tx
t")
h4nb::Config c;
c.eta = 5e-4;  c.dt_max = 1.0 / 64;  c.eps2 = 0.01 * 0.01;  c.devices = {0, 1};
h4nb::Simulation sim(p, c);
const auto e0 = sim.diagnostics();
sim.advance_to(6.3125);                                      // a multiple of dt_max
const auto e1 = sim.diagnostics();
double rel = std::abs((e1.E - e0.E) / e0.E);                 // the 1e-8 goal
if (sim.status() & h4nb::kNaN) { /* stop */ }
```

Вихідні файли ✅

`energy.dat`	`time K U E rel_err max_rel_err mean_rel_err`
`energy_periods.dat`	`period t_start t_end max_rel_err mean_rel_err complete`
`study/study.dat`	`eta dt_max t_end max_rel_err ... pair_evals wall_s pass`
`momentum.dat`	`time Px Py Pz Lx Ly Lz dP_rel dL_rel`
`timesteps.dat`	`time dt_min dt_max dt_mean mean_active r_min`
`performance.dat`	`time wall_s block_steps active pairs_per_s`
`snap_NNNNNN.snap`	`binary: header + id m x y z vx vy vz`

7. Типові проблеми

Симптом	Причина	Дія
max |dE/E| > 1e-8	η завелике	Зменште eta (спробуйте 0.0005, потім 0.00025). Запустіть дослідження η.
warning: dt < dt_min	Частинці потрібен дрібніший крок	Збільште max_level. Перевірте близькі пари.
warning: pair closer than r_close	Близьке зближення	Перевірте r_min у timesteps.dat. Розгляньте softening, якщо модель дозволяє.
non-finite value (NaN or Inf)	Хибні вхідні дані або зіткнення	Перевірте вхідні дані. Запуск зупиняється навмисно.
advance_to: t must be a multiple of dt_max	t_end, diag або snap не на сітці	Використовуйте кратні dt_max (1/64 = 0.015625).
Збірка падає з arch native	На машині збірки немає GPU	Задайте CMAKE_CUDA_ARCHITECTURES=89 або 90.
Код виходу 2 і повідомлення key=value	Невідомий ключ, хибне значення або поза діапазоном	Прочитайте повідомлення: там ключ, значення і джерело. Запустіть help=1.
Запуск 1M дуже повільний на RTX	Повільний FP64 (1/64)	RTX — для N ≤ 131 072; GH200 — для 1M.
Результати різні для різної кількості GPU	Інший порядок сум	Очікувано. Порівнюйте з записаним допуском, не побітово.

8. Обладнання

GPU	Використання	Примітка
RTX 4070 Ti (8.9)	Розробка, N ≤ 131 072	≈17 год на період для 1M (оцінка)
GH200 (9.0)	Запуски 1M і продакшн	≈18 хв на період для 1M на одній GPU (оцінка)
Нижче 7.0 (GTX 1060)	Не підтримується	Конституція 2.0.0

9. Документи

- README — `README.md`
- Constitution — `.specify/memory/constitution.md`
- Spec — `specs/001-hermite-nbody-lib/spec.md`
- Plan — `specs/001-hermite-nbody-lib/plan.md`
- Research — `specs/001-hermite-nbody-lib/research.md`
- Run configuration contract — `specs/001-hermite-nbody-lib/contracts/run-config.md`
- Tasks — `specs/001-hermite-nbody-lib/tasks.md`
- Quickstart — `specs/001-hermite-nbody-lib/quickstart.md`
- Multi-node design — `docs/multi-node-design.md`
- CUDA best practices — `docs/cuda-best-practices.md`

Згенеровано з документів репозиторію скриптом docs/guide/build_guide.py. build_guide.py